



Final Project Report:

TechQuiz: An Interactive Multi-Disciplinary Web Quiz Platform.

Web Technologies – ICT 1209

Bachelor of Information and Communication Technology (Honors)

Department of Information and Communication Technology

Faculty of Technology

Rajarata University of Sri Lanka

Students Details

- H.S.N.Edman | ITT/2024/035 | 2720
- V.P.G.G.P.Pathirana | ITT/2024/078 | 2763

Submission Date: 6th October 2026

Contents

1. Project Overview.....	ii
1.1 Problem Statement and context	1
1.2 Purpose and Objectives.....	1
1.3 Target Audience.....	1
1.4 Scope of the Application.....	1
1.5 Key Features	2
2. System Design and Architecture.....	3
2.1 System Architecture	3
2.2 Database Schema(techquiz_db).....	3
2.3 Entity-Relationship (ER) Diagram	4
2.4 Directory Structure	5
3. Implementation Details	6
3.1 Authentication Pipeline.....	6
3.2 Dynamic Quiz Engine and Countdown Timer.....	6
3.3 Score Submission and Leaderboard Querying	6
3.4 Feature Screenshots.....	7
3.4.1 User Authentication Module	7
3.4.2 Interactive Quiz Module & Countdown Timer	7
3.4.3 Real-Time Leaderboard Interface	8
3.4.4 Contact and Feedback Form	8
4. Technical Challenges and Solutions	9
5. Division of Work and Individual Contributions	9
6. Setup and Installation Guide	10
7. References.....	11

Figure 1: Entity-Relationship Diagram for techquiz_db.....	4
Figure 2: User Registration and Login Interfaces.....	7
Figure 3: Dynamic Quiz Question Interface with 30-second Countdown	7
Figure 4: SQL-driven Dynamic Leaderboard	8
Figure 5: User Inquiry Submission Form	8

Project Overview

1.1 Problem Statement and context

Conventional multiple-choice assessments and revision exercises frequently suffer from low engagement due to static, non-interactive paper interfaces. Undergraduate students and technical learners require responsive, gamified platforms that test comprehension across modern technical domains while providing instantaneous performance feedback and competitive score tracking.

1.2 Purpose and Objectives

The primary purpose of **TechQuiz** is to provide an accessible, browser-based trivia engine tailored for ICT and basic science students. The core objectives are:

- To implement a secure, session-managed authentication platform for individual user tracking.
- To design a dynamic client-side quiz module featuring category filtering and a countdown timer.
- To persist quiz attempts seamlessly to a relational database using background asynchronous calls without full-page reloads.
- To present live competitive rankings using a dynamic SQL-driven leaderboard.
- To offer an inquiry pipeline for general user feedback and support.

1.3 Target Audience

The system is built for undergraduate computing candidates, secondary school ICT students, and self-taught developers seeking structured self-assessment modules in computing, general science, and technology fundamentals.

1.4 Scope of the Application

The application encompasses client-side question rendering, timed answer validation, asynchronous score dispatching, database persistence, and ranking retrieval. It excludes external payment gateways and multi-tenant organizational administrative panels.

1.5 Key Features

1. **User Authentication & Session Security:** Secure registration and login utilizing Bcrypt password hashing and session fixation prevention via `session_regenerate_id()`.
2. **Dynamic Quiz Engine with Countdown Timer:** Interactive quiz interface featuring category selection, a 30-second automated timer per question, and instant visual validation.
3. **Asynchronous Score Persistence:** Background asynchronous `fetch()` API calls to send scores directly to the backend database without full-page reloads.
4. **Real-time Dynamic Leaderboard:** SQL-driven leaderboard displaying live rankings sorted by highest score and completion time.
5. **Feedback & Inquiry Pipeline:** Contact form allowing users to submit support queries stored directly in the relational database.

2. System Design and Architecture

2.1 System Architecture

The application follows a standard Three-Tier Architecture:

- **Presentation Tier:** Built with HTML5, CSS3, Bootstrap 5, and JavaScript (ES6+).
- **Application Tier:** Built with PHP 8 handling business logic, session validation, and database operations via PDO.
- **Data Storage Tier:** Managed by a MySQL relational database running under XAMPP.

2.2 Database Schema(techquiz_db)

The database contains three primary tables:

1. **users** Table:

- **id:** INT, PRIMARY KEY, AUTO_INCREMENT
- **username:** VARCHAR(100), NOT NULL
- **email:** VARCHAR(150), UNIQUE, NOT NULL
- **password:** VARCHAR(255), NOT NULL (Hashed with Bcrypt)
- **created_at:** TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

2. **scores** Table:

- **id:** INT, PRIMARY KEY, AUTO_INCREMENT
- **user_id:** INT, FOREIGN KEY (References users(id) ON DELETE CASCADE)
- **username:** VARCHAR(100), NOT NULL
- **category:** VARCHAR(50), NOT NULL
- **score:** INT, NOT NULL
- **played_at:** TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

3. **messages** Table:

- **id:** INT, PRIMARY KEY, AUTO_INCREMENT
- **name:** VARCHAR(100), NOT NULL
- **email:** VARCHAR(150), NOT NULL
- **message:** TEXT, NOT NULL
- **created_at:** TIMESTAMP, DEFAULT CURRENT_TIMESTAMP

2.3 Entity-Relationship (ER) Diagram

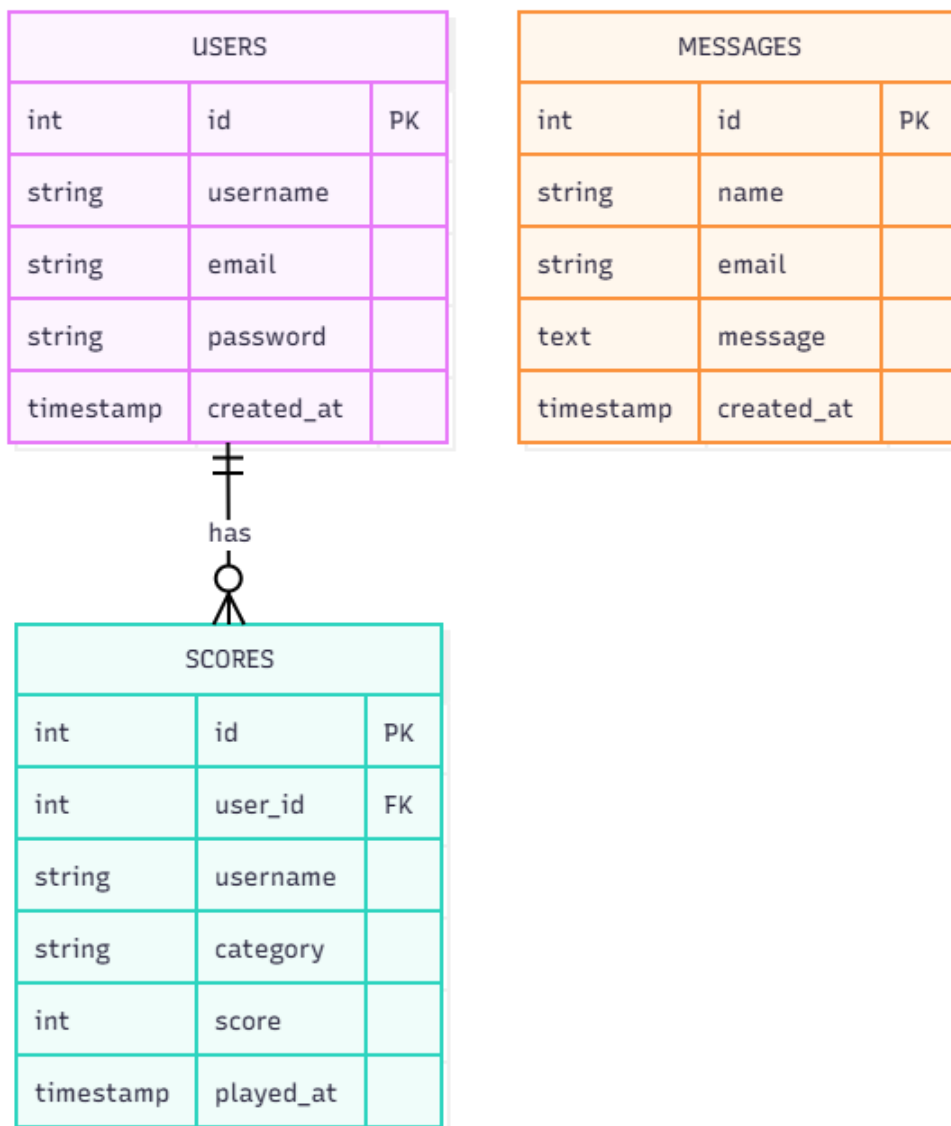


Figure 1: Entity Relationship Diagram for techquiz_db

ER Diagram Narrative Description:

- The relational database model for `techquiz_db` is designed around three main entities: users, scores, and messages.
- The users entity acts as the primary parent table holding unique user identity records.
- A 1-to-Many (1:N) relationship exists between users and scores, enforced by the `user_id` Foreign Key. This allows a single user to store multiple quiz score records while maintaining referential integrity through `ON DELETE CASCADE`.
- The messages entity operates independently to capture general user feedback and support queries submitted via the contact form.

2.4 Directory Structure

```
techquiz/
├── auth/
│   ├── login.php
│   ├── register.php
│   └── logout.php
├── css/
│   ├── style.css
│   └── leaderboard.css
├── js/
│   ├── main.js
│   └── quiz.js
├── includes/
│   ├── db.php
│   └── functions.php
├── index.php
├── dashboard.php
├── quiz.php
├── leaderboard.php
├── contact.php
├── save_score.php
├── database.sql
└── README.md
```

3. Implementation Details

3.1 Authentication Pipeline

Registration leverages PHP's `password_hash()` using `PASSWORD_BCRYPT`. During sign-in, entered passwords are verified using `password_verify()`. On successful verification, session parameters (`user_id`, `username`) are initialized, and `session_regenerate_id()` is invoked to prevent session fixation attacks.

3.2 Dynamic Quiz Engine and Countdown Timer

The quiz interface uses JavaScript arrays containing structured questions and option indices. A continuous `setInterval()` countdown enforces a strict 30-second window per question. An option selection applies instantaneous visual cues (green for correct, red for incorrect) and recalculates the user's score in real time.

3.3 Score Submission and Leaderboard Querying

Upon completing 10 questions, an asynchronous `fetch()` POST request sends the score and chosen category as a JSON payload to `save_score.php`. The backend sanitizes the request and executes a parameterized PDO query:

```
$stmt = $pdo->prepare("INSERT INTO scores (user_id, username, category, score)
VALUES (?, ?, ?, ?)");

$stmt->execute([$SESSION['user_id'], $SESSION['username'], $data['category'],
$data['score']]);
```

The leaderboard fetches real-time ranking records using:

```
SELECT username, category, score, played_at FROM scores ORDER BY score DESC,
played_at ASC LIMIT 5;
```

3.4 Feature Screenshots

3.4.1 User Authentication Module

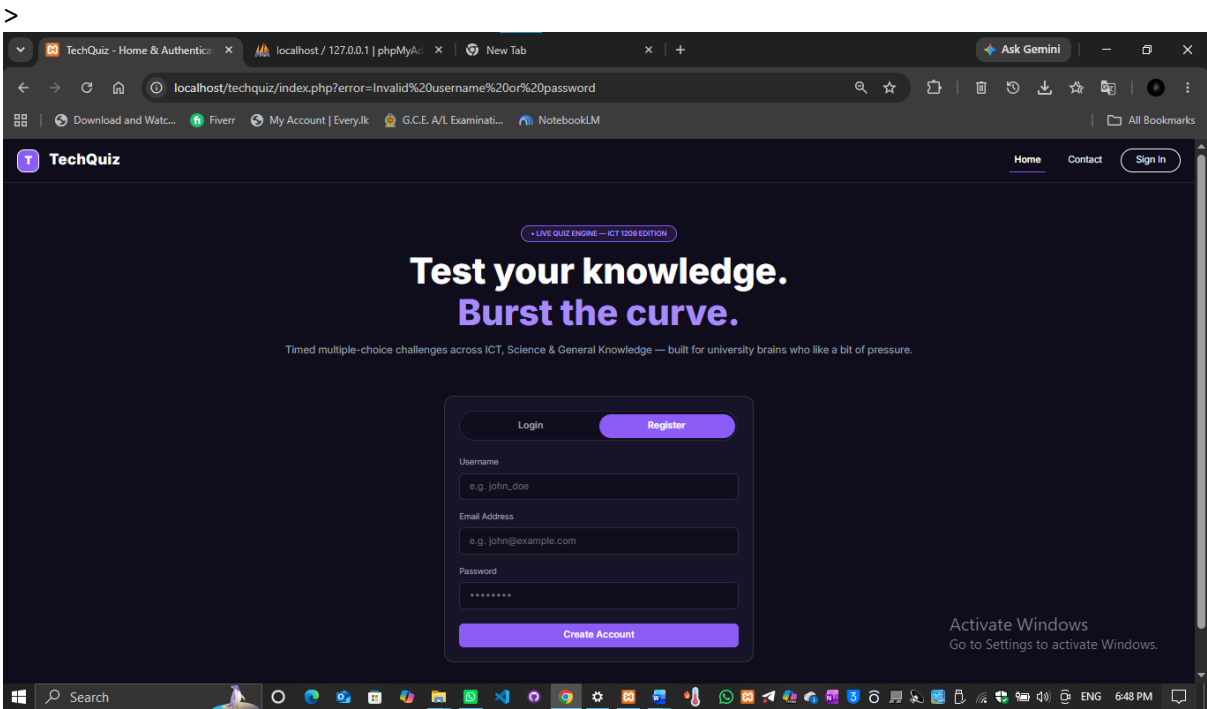


Figure 2: User Registration and Login Interfaces

3.4.2 Interactive Quiz Module & Countdown Timer

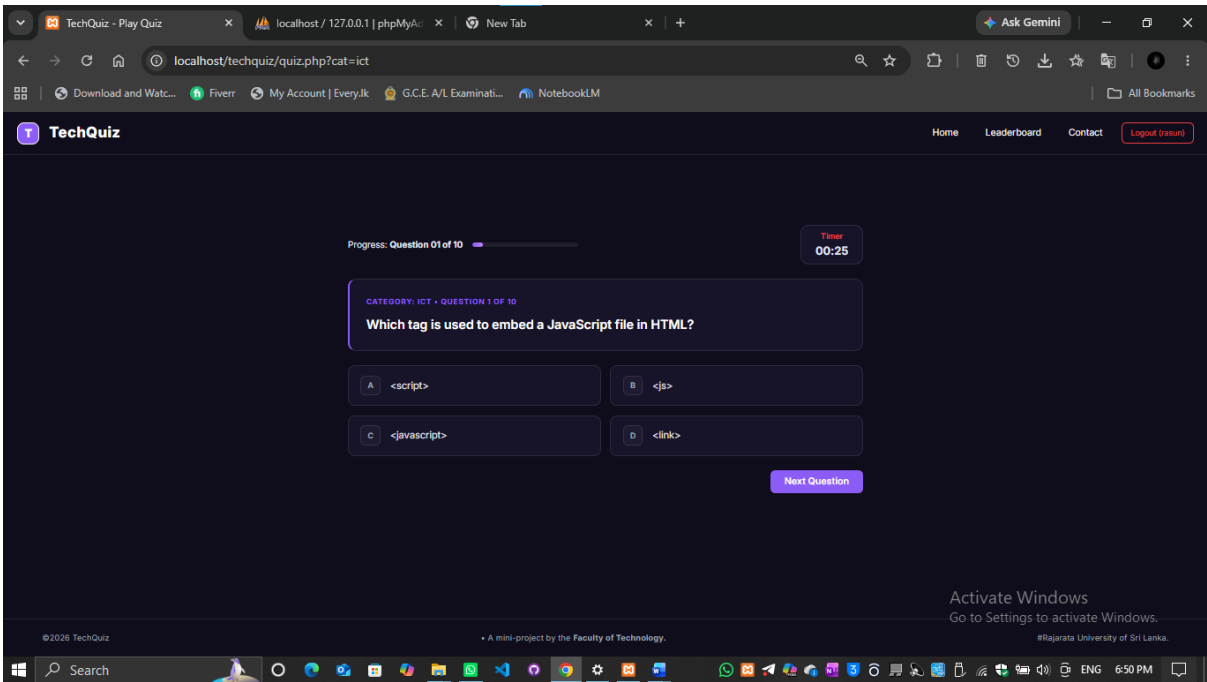


Figure 3: Dynamic Quiz Question Interface with 30 second Countdown

3.4.3 Real-Time Leaderboard Interface

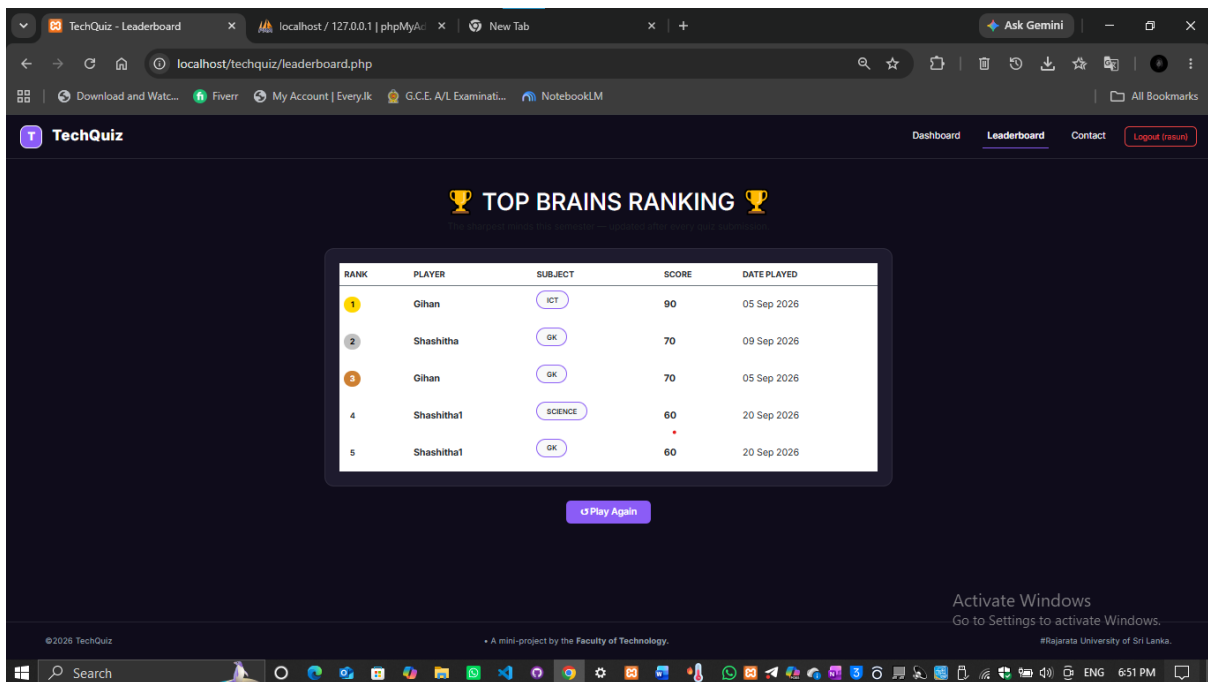


Figure 4: SQL driven Dynamic Leaderboard

3.4.4 Contact and Feedback Form

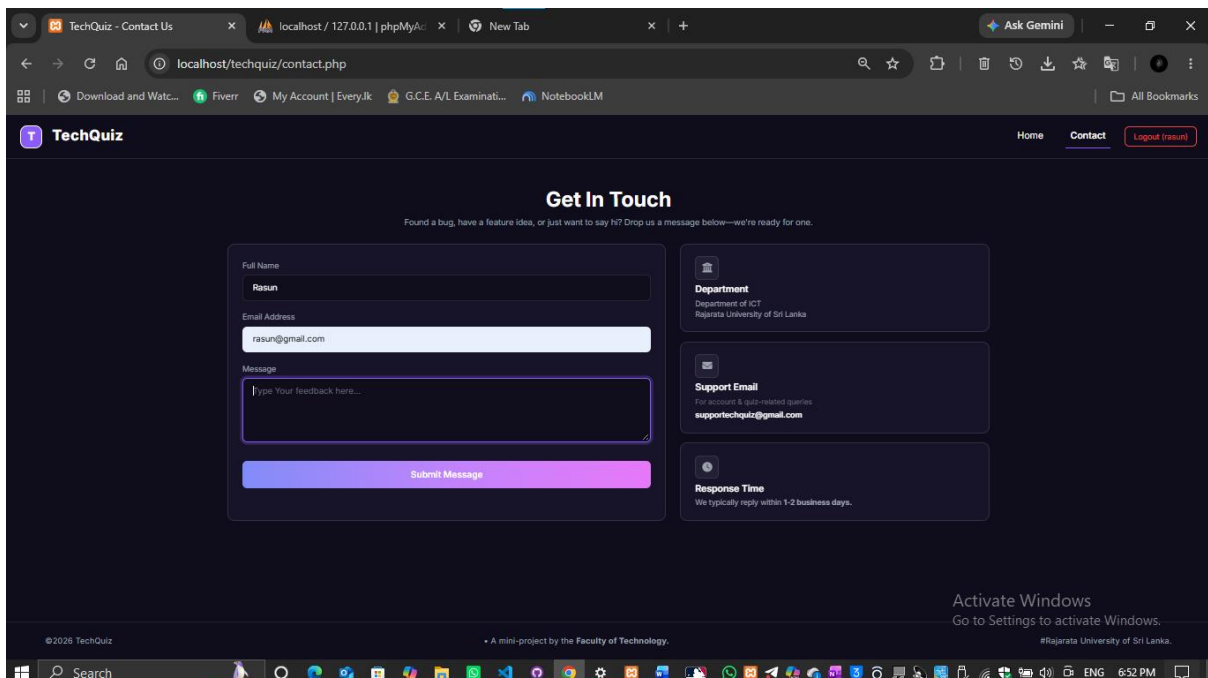


Figure 5: User Inquiry Submission Form

4. Technical Challenges and Solutions

1. Category State Loss on Page Refresh:

- **Challenge:** Selected categories were lost when transitioning between pages.
- **Resolution:** Handled category passing via URL query strings (**quiz.php?category=ict**) and parsed it via **URLSearchParams**.

2. SQL Injection Vulnerabilities:

- **Challenge:** Direct query string concatenation posed substantial security threats.
- **Resolution:** Converted all database interactions to PDO prepared statements with strict parameter binding.

3. Browser Asset Caching:

- **Challenge:** Browser caching prevented recently updated CSS and JS files from executing immediately.
- **Resolution:** Implemented dynamic timestamp query strings in file imports (e.g., **quiz.js?v=<?= time(); ?>**).

5. Division of Work and Individual Contributions

- **H. S. N. Edman (Reg: ITT/2024/035 | Index: 2720):**
 - Relational schema creation and foreign key definitions (**users, scores, messages**).
 - Quiz logic development, 30-second interval timer integration, and option feedback in JavaScript.
 - Asynchronous backend API development (**save_score.php**) and leaderboard dynamic queries.
- **V. P. G. G. P. Pathirana (Reg: ITT/2024/078 | Index: 2763):**
 - Responsive frontend page layouts using Bootstrap 5 and custom dark stylesheet.
 - User authentication flow, session security, and logout controls.
 - Contact inquiry submission routing and documentation preparation (README.md).

6. Setup and Installation Guide

- Clone the repository into your local web root directory:
- `git clone [https://github.com/GihanGiwika/ICT-1209---Mini-Project.git]`(<https://github.com/GihanGiwika/ICT-1209---Mini-Project.git>)
techquiz
- Start the **Apache** and **MySQL** modules using the XAMPP Control Panel.
- Navigate to <http://localhost/phpmyadmin/> in your browser and create a database named techquiz_db.
- Import the provided database.sql script into the database.
- Open your web browser and access the application at <http://localhost/techquiz/>.

7. References

- **PHP Documentation Group (2026)** PHP Manual: Password Hashing & PDO Prepared Statements. Available from: <https://www.php.net/docs.php>
- **Bootstrap Team (2026)** Bootstrap v5.3 Responsive Front-End Framework Component Documentation. Available from: <https://getbootstrap.com/>
- **MDN Web Docs (2026)** Using the Fetch API and Asynchronous JavaScript. Mozilla Foundation. Available from: <https://developer.mozilla.org/>
- **MySQL AB (2026)** MySQL 8.0 Reference Manual: Foreign Key Constraints & InnoDB Storage Engine. Oracle Corporation. Available from: <https://dev.mysql.com/doc/>